

OS_2024

1 Synchronisation mit Semaphoren (30)

Verwenden Sie Semaphore zur Lösung des folgenden Synchronisationsproblems zwischen den zyklischen Prozessen *A*, *B*, *C*, *D* und *E*. Prozess *A* schreibt immer wieder Daten auf ein Shared Memory, in dem Platz für einen Datensatz ist. *B*, *C* und *D* lesen jeden von *A* geschriebenen Datensatz vom Shared Memory und verarbeiten diesen. Nach der Fertigstellung der Verarbeitung durch *B*, *C* und *D* wird im Prozess *E* die Funktion *finish()* ausgeführt. Beachten Sie folgendes:

- Jeder Datensatz, der von *A* mittels *write_ShM()* auf das Shared Memory geschrieben wird, muss von *B*, *C* und *D* genau einmal mittels *read_ShM()* gelesen werden.
- *B*, *C* und *D* müssen einzelne Datensätze parallel verarbeiten können.
- Nachdem *B*, *C* und *D* vom Shared Memory gelesen haben, verwenden sie die Funktion *use_data()*, um die vom Shared Memory gelesenen Daten zu verarbeiten.
- Wenn die drei Prozesse mit der Verarbeitung der Daten fertig sind, ruft *E* die Funktion *finish()* auf. Danach beginnt die nächste Verarbeitungsrunde, d.h. *A* schreibt wieder auf das Shared Memory.

Ergänzen Sie die Codestücke für die Prozesse *A*, *B*, *C*, *D* und *E* und geben Sie passende Initialisierungen an.

Initialisierungen

```
init(Astart, 1);
init(Bstart, 0);
init(Cstart, 0);
init(Dstart, 0);

init(BDone, 0);
init(CDone, 0);
init(DDone, 0);
```

```
/** Prozess A **/
```

```
for(;;) {
```

```
    wait(Astart);
```

```
    writeShm(data);
```

```
    signal(Bstart);
    signal(Cstart);
    signal(Dstart);
```

```
/** Prozess B **/
```

```
for(;;) {
```

```
    wait(Bstart);
```

```
    readShM(&data);
```

```
    use_data(data);
```

```
/** Prozess C **/
```

```
for(;;) {
```

```
    wait(Cstart);
```

```
    readShM(&data);
```

```
    use_data(data);
```

```

                                signal(BDone);
                                signal(CDone);

                                }
                                }
                                }

    /** Prozess D **/
    for(;;) {

        wait(Dstart);

        readShM(&data);

        use_data(data);

        signal(DDone);

    }

    /** Prozess E **/
    for(;;) {

        wait(BDone);
        wait(CDone);
        wait(DDone);

        finish();

        signal(Astart);

    }

```

2 Memory Management (30)

Für einen Arbeitsspeicher mit Paging ist eine Seitenzugriffsfolge gegeben. Für diese Zugriffsfolge sollen das Working Set sowie die Belegung der Pages nach jedem Speicherzugriff bei Verwendung unterschiedlicher Seiten-Ersetzungsstrategien beschrieben werden.

Working Set: Geben Sie in jeder Spalte der Tabelle das Working Set $W(D, t)$ mit $D = 5$ nach jedem in der Kopfzeile gegebenen Seitenzugriff an.

$W(5, t)$	A	B	C	D	B	E	A	F	D	E	B	D	B	F	E	D	F	C
	A	B A	C B A	D C B A	B D C A	E D C	A E B D C	F A E B D	D F A E B	E D F A	B E D F A	D B E F	B D E F	F B D E	E C D D	D E F B	F D E B	C F D E

Page Replacement: Geben Sie nun in den Spalten der Tabellen die Speicherinhalte für jeden Frame nach jedem in der Kopfzeile angegebenen Seitenzugriff bei Verwendung der Replacement Strategien LRU bzw. FIFO an. Kennzeichnen Sie in der letzten, mit *PF* markierten Zeile das Auftreten von Page Faults. Die anfängliche Belegung des Arbeitsspeichers ist in der Tabelle eingetragen. Die Seiten wurden in der Reihenfolge *E, A, B, C* in den Speicher geladen.

LRU	A	B	C	D	B	E	A	F	D	E	B	D	B	F	E	D	F	C
0	E	E	C	D	D	D	D	F	F	F	F	F	F	F	F	F	F	F
1	A	A	A	A	A	E	E	E	E	E	E	E	E	E	E	E	E	E
2	B	B	B	B	B	B	B	B	D	D	D	D	D	D	D	D	D	D
3	C	C	C	C	C	C	A	A	A	A	B	B	B	B	B	B	B	C
PF	-			X		X	X	X	X		X							X

FIFO	A	B	C	D	B	E	A	F	D	E	B	D	B	F	E	D	F	C
0	E	E	E	D	D	D	D	D	D	D	B	B	B	B	B	B	B	B
1	A	A	A	A	A	E	E	E	E	E	E	D	D	D	D	D	D	D
2	B	B	B	B	B	B	A	A	A	A	A	A	A	A	E	E	E	E
3	C	C	C	C	C	C	C	F	F	F	F	F	F	F	F	F	F	C
PF	-			X		X	X	X			X	X			X			X

Fragen:

1. Threadimplementierung (4 Punkte)

Was versteht man unter einem *Kernel Level Thread* und unter einem *User Level Thread*?

Beschreiben Sie die beiden Arten der Threadimplementierung und charakterisieren Sie deren Unterschiede.

KLT:

- Vom Kernel gesehen und mittels Kernel Thread API verwaltet
- Thread switching durch Kernel
- blockieren einzelner Threads möglich
- Multicore -> KLT kann in zweitem Kern ausgeführt werden

ULT:

- Für Kernel unsichtbar
 - Thread switching durch Thread Library im User Mode möglich
 - applikationsspezifisches Scheduling
 - Thread geblockt -> ganzer Prozess geblockt
 - Synchronisation in User Mode
 - kein Ausführen auf mehreren Kernen gleichzeitig
-

2. Deadlocks (4 Punkte)

Nennen Sie die Bedingungen für das Eintreten eines Deadlocks und erklären Sie diese.

- Mutex -> gewisse Bereiche erlauben nur eine gewisse Anzahl an Prozessen
 - No preemption -> keine Ressourcen werden weggenommen
 - Hold and Wait -> Prozess kann Ressourcen blockieren und auf andere Ressourcen warten
 - Circular Wait -> Prozess wartet auf anderen Prozess der aber auf den Prozess wartet der auf ihn wartet
-

3. Relocation (2 Punkte)

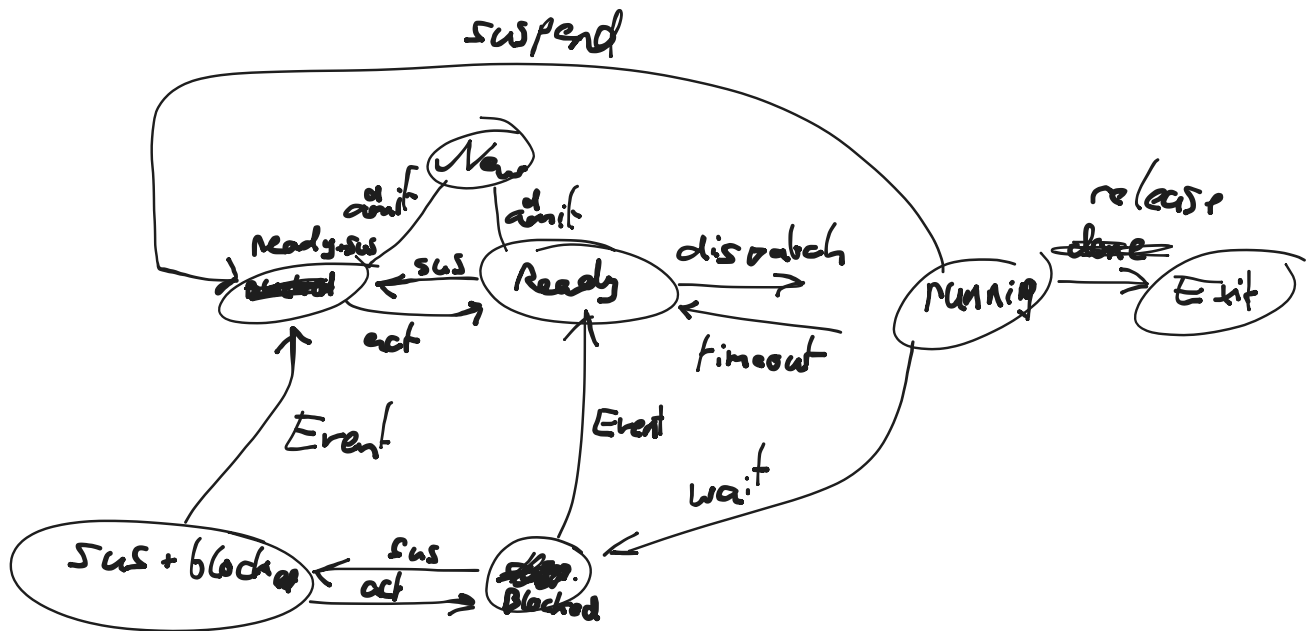
Was versteht man unter dem Begriff *Relocation*? Wofür ist Relocation von Bedeutung?

- Relocation ist die Umsetzung von logischen (virtuellen) Adressen, die das Programm verwendet, auf physische Adressen im RAM (meist durch die MMU).
 - Relocation ist der Fall, wenn man nach beispielsweise einem Swapping (von RAM in Sekundärspeicher) Daten die vor dem Swap bei physischer Adresse A hatte jetzt bei Adresse B findet.
-

4. Zustandsdiagramm eines Prozesses (5 Punkte)

Zeichnen Sie das Zustandsdiagramm eines Prozesses, beginnend mit der Entstehung, bis zur

Beendigung des Prozesses. Geben Sie die Namen der Zustände an und beschreiben Sie kurz jeden Zustand und Zustandsübergang.



5. Pfadnamen (2 Punkte)

Erklären Sie die Begriffe *absoluter Pfadname* und *relativer Pfadname* und geben Sie jeweils ein Beispiel an.

Absoluter Pfad startet bei root folder und beginnt mit /

- /etc/file.txt

Relativer Pfad startet bei Pfad in dem man sich befindet

- folder/file.txt

6. Logische Struktur des I/O Systems (4 Punkte)

Mit welcher logischen Struktur des I/O Systems versucht man, bei der Realisierung von I/O Funktionen sowohl ein einheitliches Programmierinterface, als auch eine möglichst gerätespezifische Ansteuerung zu erreichen?

Mit der Aufteilung in die 3 Schichten:

- Logische IO: einheitliches Interface für Programmierer (open, read...)
- Geräte IO (logische Befehle in gerätespezifische Befehlssequenzen umgewandelt)
- Scheduling und Kontrolle (Hardware integration)

7. CPU Scheduling Strategien (4 Punkte)

Beschreiben Sie die folgenden Strategien für das CPU Scheduling und vergleichen Sie deren Eigenschaften:

- *First Come First Serve*
- *Round Robin*
- *Feedback Scheduling*

FCFS:

- Prozesse werden in der Reihenfolge bearbeitet in der sie kommen...
- Non-Preemptive
- Heißt auch dass Lange Prozesse verhältnismäßig wenig warten

RR:

- Jeder Prozess hat eine genaue Zeitanzahl Zeit in der der Prozess die CPU hat
- Wenn Zeit aufgebraucht wird Prozess pausiert und Prozess kommt zurück in die Queue
- Preemptive

Feedback Scheduling:

- Neue Prozesse starten mit **hoher Priorität**. Wenn sie ihre Zeit aufbrauchen (also CPU-lastig sind), werden sie **abgestraft** und in eine Queue mit niedrigerer Priorität (aber oft längerer Zeitscheibe) verschoben.
 - verschiedene Priority Queues
 - Preemptive
-

8. Blocking vs. Non-Blocking I/O (3 Punkte)

Was versteht man unter *Blocking* bzw. *Non-Blocking I/O*? Beschreiben Sie die beiden Arten, I/O-Operationen durchzuführen.

Blocking IO -> Prozess wartet bis IO Operation fertig ist und macht dann erst weiter

Non-Blocking -> Der Systemaufruf kehrt **sofort (immediately)** zurück, auch wenn die I/O-Operation noch nicht fertig ist. Der Prozess wird **nicht** blockiert. Der Prozess muss meistens später erneut fragen (Polling) oder bekommt eine Benachrichtigung (Asynchronous I/O), um das Ergebnis zu verarbeiten.